

Observer Design Pattern

University Project: Case Study 3

Order Tracking System

Prepared by:

Mohammad Shoman

Ibrahim Al-Balawi

Ahmed Al-Ajmi



Why do we need an Observer?



Tight Coupling

Hardcoding notification logic inside the Order class makes it difficult to maintain and extend with new tracking channels.



Resource Waste

Systems constantly "polling" the server to ask for status updates consumes excessive CPU and network bandwidth.



Rigid Architecture

Adding a new notification channel (e.g., WhatsApp) requires modifying the core business logic of the order lifecycle.



Scalability Issues

Manually managing thousands of concurrent updates leads to performance bottlenecks and potential system crashes.

The Observer Solution

Push-Based Updates

We move from "asking for data" to "broadcasting changes".
The Subject automatically pushes state updates to all listeners.

Decoupled Logic

The Order object doesn't know who is listening. It communicates through an abstraction, ensuring a clean separation of concerns.

Dynamic Subscription

Subsystems can register or unregister at runtime without affecting the core order processing engine.

Observer vs State Pattern

		
Primary Focus	One-to-Many Notifications	Internal Logic Encapsulation
Coupling	Loose (via Interface)	Tight (Internal Transitions)
Flexibility	High (Add/Remove Observers)	Medium (Finite State Machine)
Use Case	External Subsystem Updates	Internal Behavioral Change

The Observer Pattern is superior for broadcasting updates to independent, heterogeneous subsystems.

System Architecture

Subject (Order)

The core data model. It maintains the shipment state and manages the list of subscribers.

Observer (Interface)

The contract. It defines a standard `update()` method that all tracking systems must implement.

Concrete Observers

Independent modules like **CustomerApp** or **SMSService** that react to fulfillment changes.

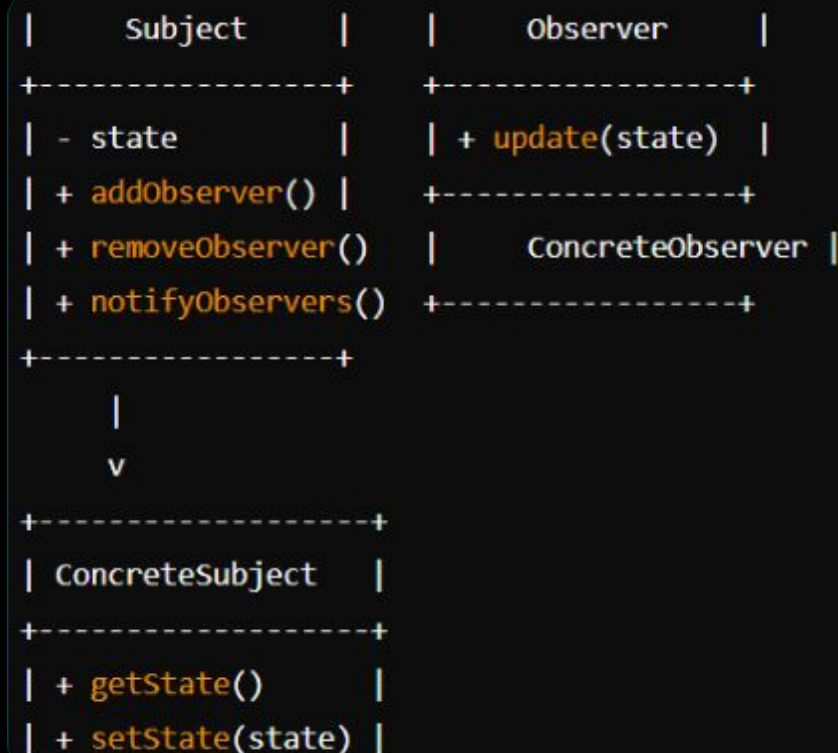
Java Implementation Details

1. Recursive Broadcast

When `setStatus()` is called, the Subject iterates through its internal list and invokes the `update()` method on each subscriber.

```
for (Observer obs : observers) {
    obs.update(status);
}
```

This ensures that presentation and communication logic are never mixed with core business rules.



Execution & Final Output



System Real-Time Log

```
> Order ORD-9988 Placed.  
> Status: OUT_FOR_DELIVERY  
[Customer iOS App] Notification: Order is now  
OUT_FOR_DELIVERY  
[SMS System] Sent: Your package is with the courier.
```

Success: 100% decoupling achieved with zero resource waste from polling.